

Le rapport du projet SE "Embedded AI on FPGA using High Level Synthesis"

Table des matières

I.	Introduction / Position du problème	2
II.	Méthodologie	2
1.	Outils et environnement de développement	2
2.	Flot de développement du projet	3
3.	Versions étudiées	3
4.	Méthodes de mesure des performances	3
III.	Implémentation du réseau de neurones convolutif	3
1.	Apprentissage du réseau et génération des poids	3
2.	Implémentation en langage C – Version flottante	4
3.	Conversion en virgule fixe	4
4.	Gestion des poids pour la synthèse HLS	5
5.	Synthèse matérielle avec Vivado HLS	5
6.	Intégration système avec SDSoC	5
IV.	Validation et performances	6
1.	Validation fonctionnelle	6
2.	Performances de la version logicielle (SW)	6
3.	Résultats de synthèse Vivado HLS	6
4.	Performances sur la carte ZedBoard (SDSoC)	7
5.	Analyse comparative	8
V.	Conclusion	8

I. Introduction / Position du problème

L'intelligence artificielle embarquée connaît un intérêt croissant, notamment pour des applications nécessitant des traitements intensifs tout en respectant des contraintes strictes de performances, de consommation énergétique et de latence. Parmi ces applications, la reconnaissance d'images et de motifs constitue un cas d'étude représentatif, largement utilisé pour l'évaluation de techniques d'accélération matérielle.

Dans ce contexte, les réseaux de neurones convolutifs (Convolutional Neural Networks – CNN) sont aujourd'hui des modèles de référence pour la classification d'images. Cependant, leur complexité algorithmique rend leur exécution coûteuse sur des processeurs généralistes, en particulier dans des systèmes embarqués aux ressources limitées.

L'objectif de ce projet est d'implémenter et d'accélérer un réseau de neurones convolutif de type LeNet pour la classification des chiffres manuscrits de la base de données MNIST, en s'appuyant sur une plateforme hétérogène CPU/FPGA. L'accélération matérielle est réalisée à l'aide de la synthèse de haut niveau (High Level Synthesis – HLS), qui permet de générer des accélérateurs matériels à partir de code en langage C.

Le projet vise plus précisément à comparer trois versions d'implémentation du réseau :

- Une version purement logicielle exécutée sur le processeur ARM Cortex-A9.
- Une version matérielle séquentielle (HW_SEQ) synthétisée sur FPGA sans parallélisation explicite.
- Une version matérielle parallélisée (HW_PAR) exploitant les capacités de parallélisme du FPGA à l'aide de pragmas HLS.

L'étude porte à la fois sur les performances en termes de temps d'exécution et de nombre de cycles d'horloge, ainsi que sur l'utilisation des ressources matérielles du FPGA (BRAM, DSP, LUT, registres). L'objectif final est de mettre en évidence l'intérêt et les limites de l'accélération matérielle pour ce type d'application embarquée.

II. Méthodologie

1. Outils et environnement de développement

Le projet a été réalisé sur une plateforme hétérogène CPU/FPGA basée sur une carte ZedBoard intégrant un SoC Xilinx Zynq-7000, composé d'un processeur ARM Cortex-A9 et d'une logique programmable FPGA.

Les principaux outils utilisés dans le cadre de ce projet sont :

- Python avec TensorFlow/Keras pour l'apprentissage du réseau de neurones.
- GCC pour la compilation des versions logicielles en langage C
- Vivado HLS pour la synthèse de haut niveau des accélérateurs matériels.
- SDSoC pour l'intégration des accélérateurs dans un système Linux embarqué.

Le jeu de données MNIST a été utilisé pour l'entraînement et la validation du réseau de neurones convolutif.

2. Flot de développement du projet

Le développement du projet a été réalisé de manière progressive. Dans un premier temps, un modèle de réseau de neurones convolutif de type LeNet a été entraîné en Python afin d'obtenir des poids optimisés pour la classification des chiffres manuscrits MNIST.

Dans un second temps, une implémentation complète du réseau a été développée en langage C à partir des poids générés. Cette implémentation a été validée fonctionnellement en virgule flottante, puis convertie en virgule fixe afin d'être compatible avec une implémentation matérielle sur FPGA.

Une fois la version fixe validée, les fonctions critiques du réseau ont été synthétisées à l'aide de Vivado HLS afin de générer des accélérateurs matériels. Enfin, ces accélérateurs ont été intégrés dans un système embarqué complet à l'aide de SDSoc.

3. Versions étudiées

Trois versions du réseau ont été étudiées dans ce projet :

- Une version purement logicielle (SW), exécutée sur le processeur ARM Cortex-A9.
- Une version matérielle séquentielle (HW_SEQ), synthétisée sur FPGA sans parallélisation explicite.
- Une version matérielle parallélisée (HW_PAR), exploitant les capacités de parallélisme du FPGA à l'aide de pragmas HLS.

Ces différentes versions permettent de comparer l'impact de l'accélération matérielle et du parallélisme sur les performances et l'utilisation des ressources FPGA.

4. Méthodes de mesure des performances

Les performances ont été évaluées à l'aide de deux indicateurs principaux.

D'une part, le nombre de cycles d'horloge a été extrait des rapports de synthèse générés par Vivado HLS, ce qui permet de comparer les versions matérielles indépendamment de la fréquence d'horloge.

D'autre part, les temps d'exécution réels ont été mesurés sur la carte ZedBoard à l'aide de fonctions de mesure de temps sous Linux, notamment gettimeofday.

III. Implémentation du réseau de neurones convolutif

1. Apprentissage du réseau et génération des poids

Le point de départ du projet consiste en l'apprentissage d'un réseau de neurones convolutif de type LeNet à l'aide d'un script Python fourni par l'enseignant. Ce script repose sur les bibliothèques TensorFlow et Keras et permet d'entraîner le réseau sur la base de données MNIST.

L'apprentissage est réalisé sur plusieurs époques, montrant une convergence progressive du modèle avec une augmentation de la précision et une diminution de la fonction de perte. À l'issue de l'apprentissage, les poids du réseau sont sauvegardés dans un fichier au format HDF5. Ce fichier contient l'ensemble des poids et biais nécessaires à l'exécution de l'inférence du réseau.

La précision obtenue à la fin de l'apprentissage est supérieure à 98 %, ce qui valide la qualité du modèle avant son implémentation embarquée.

2. Implémentation en langage C – Version flottante

À partir des poids générés, une implémentation complète du réseau de neurones a été développée en langage C. Le fichier principal `lenet_cnn_float.c`, fourni par l'enseignant, permet d'orchestrer l'exécution de l'ensemble du réseau et de tester son bon fonctionnement.

Trois fichiers sources principaux ont été développés :

- `fc.c`, regroupant les fonctions correspondant aux deux couches entièrement connectées,
- `pool.c`, regroupant les fonctions correspondant aux deux couches de pooling,
- `conv.c`, regroupant les fonctions correspondant aux deux couches de convolution.

Le développement a été effectué de manière progressive. Les couches entièrement connectées ont été implémentées en premier, car elles sont les plus simples à comprendre et à déboguer. Les couches de pooling ont ensuite été développées, avant de passer aux couches de convolution, qui constituent la partie la plus complexe du réseau.

Une fois ces fichiers implémentés, l'exécution du réseau en virgule flottante a été validée. Les résultats obtenus montrent un taux de reconnaissance proche de celui obtenu lors de l'apprentissage en Python, confirmant la validité fonctionnelle de l'implémentation.

3. Conversion en virgule fixe

Afin de rendre le code compatible avec une implémentation matérielle sur FPGA, l'ensemble des calculs du réseau a été converti en virgule fixe. Cette conversion concerne les couches de convolution, de pooling et les couches entièrement connectées.

La version fixe du réseau est testée à l'aide du fichier `lenet_cnn_fixed.c`. Les tests réalisés montrent un taux de succès d'environ **97 %**, ce qui reste très proche de la version flottante. Cette légère perte de précision est attendue et acceptable dans le cadre d'une implémentation embarquée.

La validation de cette version confirme que la représentation en virgule fixe constitue un bon compromis entre précision et compatibilité matérielle.

4. Gestion des poids pour la synthèse HLS

L'utilisation de Vivado HLS impose certaines contraintes, notamment l'impossibilité d'exploiter directement des fichiers de poids au format HDF5 lors de la synthèse. Pour contourner cette limitation, une solution spécifique a été mise en place.

Des instructions d'affichage ont été ajoutées dans le fichier `utils.c`, chargé de la lecture des poids depuis le fichier HDF5. Les valeurs affichées ont ensuite été récupérées manuellement afin de constituer un fichier d'en-tête `weights.h`, contenant l'ensemble des poids du réseau.

Contrairement à une approche classique basée sur des tableaux statiques, les poids ont été implémentés sous forme de fonctions retournant les valeurs correspondantes. Cette méthode entraîne un temps de compilation plus long, mais n'impacte pas significativement les performances lors de la synthèse HLS et de l'exécution du design.

Grâce à cette approche, le réseau en virgule fixe peut être synthétisé et exécuté sans dépendance externe au format HDF5.

5. Synthèse matérielle avec Vivado HLS

Une fois la version fixe validée, le projet a été importé dans Vivado HLS afin de générer des accélérateurs matériels. Plusieurs essais ont été réalisés en modifiant la fonction à accélérer (réseau complet, couches de convolution, couches de pooling, couches entièrement connectées) afin d'analyser l'impact de chaque composant sur les performances et l'utilisation des ressources.

Dans un premier temps, une version matérielle séquentielle (`HW_SEQ`) a été synthétisée sans utilisation de pragmas de parallélisation. Cette version permet d'obtenir une première estimation des performances et de l'occupation des ressources FPGA.

Dans un second temps, des pragmas HLS de parallélisation (pipeline) ont été progressivement ajoutés. À chaque itération, l'objectif était de réduire le nombre de cycles tout en respectant les contraintes de ressources, en évitant notamment de dépasser 100 % d'utilisation des BRAM, DSP, LUT et registres.

6. Intégration système avec SDSoC

La dernière étape du projet consiste à intégrer les différentes versions du réseau de neurones dans un système complet à l'aide de l'environnement SDSoC, permettant l'exécution de l'application sous Linux sur la carte ZedBoard.

La version purement logicielle du réseau, sans accélérateur matériel, a été exécutée avec succès sur la carte. Les résultats obtenus sont fonctionnels et montrent un taux de reconnaissance conforme aux versions précédentes. Comme attendu, le temps d'exécution est élevé, ce qui met en évidence les limites d'une implémentation purement logicielle pour ce type d'application.

Les versions intégrant un accélérateur matériel ont également été exécutées sur la carte ZedBoard. Une première version sans pragmas de parallélisation, puis une version optimisée à

l'aide de pragmas HLS, ont été testées. Dans les deux cas, l'exécution est fonctionnelle et produit des résultats corrects en termes de classification et de taux de succès.

Les différences observées entre ces versions concernent principalement les performances. La version matérielle sans parallélisation présente un temps d'exécution supérieur à la version logicielle, tandis que la version parallélisée permet une réduction significative du temps de traitement. Ces résultats sont cohérents avec les estimations issues de la synthèse Vivado HLS et confirment l'intérêt de l'accélération matérielle associée à une parallélisation adaptée.

IV. Validation et performances

1. Validation fonctionnelle

Les différentes versions du réseau de neurones ont été validées fonctionnellement à chaque étape du projet.

Pour chacune des implémentations (logicielle et matérielles), le réseau est capable de classifier correctement les images de la base MNIST et d'afficher les probabilités de sortie ainsi que la classe prédite.

Les résultats obtenus montrent un taux de reconnaissance proche de **97 %**, aussi bien pour la version logicielle que pour les versions matérielles, ce qui confirme que les optimisations matérielles et la conversion en virgule fixe n'altèrent pas significativement la précision du réseau.

2. Performances de la version logicielle (SW)

La version purement logicielle du réseau, exécutée sur le processeur ARM Cortex-A9 de la carte ZedBoard, a été mesurée à l'aide de la fonction `gettimeofday`.

Le temps total de traitement observé pour l'exécution complète du réseau sur l'ensemble du jeu de test MNIST est de :

- **Temps total SW : 3390 secondes**

Ce temps d'exécution très élevé met en évidence les limites d'une implémentation purement logicielle pour ce type d'algorithme intensif en calcul, et justifie l'utilisation d'une accélération matérielle.

3. Résultats de synthèse Vivado HLS

Les versions matérielles ont été analysées à l'aide des rapports de synthèse générés par Vivado HLS. Les résultats suivants correspondent aux versions sans parallélisation (HW_SEQ) et avec parallélisation (HW_PAR).

Latence en nombre de cycles

Version	Nombre de cycles
HW_SEQ	8 300 525 cycles

Version	Nombre de cycles
HW_PAR	980 991 cycles

L'ajout de pragmas de parallélisation permet une réduction très significative du nombre de cycles, avec un facteur d'accélération supérieur à **8×** entre les versions HW_SEQ et HW_PAR.

Utilisation des ressources FPGA

Version	BRAM	DSP	FF	LUT
HW_SEQ	50 (17 %)	12 (5 %)	1 %	6 %
HW_PAR	65 (23 %)	67 (30 %)	14 %	62 %

Ces résultats montrent que la parallélisation augmente fortement l'utilisation des ressources matérielles, en particulier les DSP et les LUT, ce qui est attendu dans une architecture exploitant le parallélisme spatial du FPGA.

4. Performances sur la carte ZedBoard (SDSoC)

Les différentes versions du réseau de neurones ont été exécutées sur la carte ZedBoard afin d'évaluer les performances en conditions réelles d'exécution dans un environnement Linux embarqué.

Les temps d'exécution mesurés pour chaque version sont récapitulés dans le tableau suivant.

Temps d'exécution mesurés

Version	Temps d'exécution
SW (ARM Cortex-A9)	3390 s
HW_SEQ	4120 s
HW_PAR	145 s

La version matérielle séquentielle (HW_SEQ), bien qu'exécutée à l'aide d'un accélérateur matériel, présente un temps d'exécution supérieur à celui de la version purement logicielle. Ce comportement s'explique par l'absence de parallélisation explicite et par le surcoût lié aux communications entre le processeur et l'accélérateur matériel.

En revanche, la version matérielle parallélisée (HW_PAR) permet une réduction significative du temps de traitement. L'exploitation du parallélisme offert par le FPGA permet d'améliorer notablement les performances globales du système, tout en conservant un comportement fonctionnel identique aux autres versions.

5. Analyse comparative

L'analyse comparative des différentes versions met en évidence l'impact déterminant de la parallélisation dans un contexte d'accélération matérielle sur FPGA.

La version logicielle, exécutée sur le processeur ARM Cortex-A9, constitue une référence fonctionnelle mais souffre de temps d'exécution très élevés, en raison du caractère fortement intensif en calcul du réseau de neurones convolutif.

La version matérielle séquentielle représente une première étape vers l'accélération matérielle. Toutefois, en l'absence de parallélisme, les gains potentiels du FPGA ne sont pas exploités. De plus, les échanges de données entre le processeur et l'accélérateur induisent un surcoût qui pénalise les performances globales, conduisant à un temps d'exécution supérieur à celui de la version logicielle.

Enfin, la version matérielle parallélisée exploite pleinement le parallélisme spatial du FPGA. Cette approche permet de réduire fortement le temps d'exécution par rapport aux deux autres versions, au prix d'une augmentation de l'utilisation des ressources matérielles. Ces résultats confirment l'intérêt de l'association entre synthèse de haut niveau et parallélisation pour l'accélération efficace de réseaux de neurones convolutifs dans des systèmes embarqués.

V. Conclusion

Ce projet avait pour objectif d'étudier l'implémentation et l'accélération d'un réseau de neurones convolutif pour la classification d'images MNIST sur une plateforme embarquée CPU/FPGA, en s'appuyant sur la synthèse de haut niveau.

Une implémentation complète du réseau a tout d'abord été réalisée en langage C et validée en virgule flottante, puis convertie en virgule fixe afin de répondre aux contraintes d'une implémentation matérielle. Les résultats obtenus montrent que cette conversion n'entraîne qu'une légère perte de précision, avec un taux de reconnaissance d'environ 97 %, ce qui reste tout à fait satisfaisant pour une application embarquée.

L'étude des différentes versions du réseau a permis de mettre en évidence l'impact de l'accélération matérielle et du parallélisme sur les performances. La version purement logicielle, bien que fonctionnelle, présente des temps d'exécution très élevés. La version matérielle séquentielle constitue une première étape vers l'accélération, mais ne permet pas d'exploiter pleinement les capacités du FPGA. En revanche, l'ajout de pragmas de parallélisation permet de tirer parti du parallélisme spatial du FPGA et d'obtenir une réduction significative du temps de traitement, au prix d'une augmentation de l'utilisation des ressources matérielles.

Ce projet a ainsi permis de mieux comprendre les enjeux liés à l'implémentation de réseaux de neurones convolutifs sur FPGA, ainsi que les compromis nécessaires entre performances, précision et consommation de ressources.

Fait par :

Meriem NAOUI

Laura BOIVIN

Encadrant :

Sebastien BILAVARN